

中山大學
SUN YAT-SEN UNIVERSITY

ANNs | HW2

基于 BiLSTM-Attention 与 Transformer 的 中文短新闻主题分类

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 潘炎

起 始 日 期: 2026 年 5 月 24 日

结 束 日 期: 2026 年 6 月 16 日

实 验 地 点: B204

目录

1 实验内容 **2**
1.1 任务描述 . . . 2

2 实验项目结构 **2**

3 环境与使用说明 **2**
3.1 硬件与软件 . . . 2
3.2 运行指令 . . . 2

4 分析设计 **3**
4.1 数据预处理 . . . 3
4.2 BiLSTM + Bahdanau 注意力 . . . 5
4.3 Transformer Encoder + 均值池化 . . . 5
4.4 训练设置 . . . 5
4.5 抗过拟合策略的设计原理 . . . 6

5 核心代码 **6**
5.1 Bahdanau 加性注意力机制 (src/models.py) . . . 6
5.2 BiLSTM-Attention 前向传播 (src/models.py) . . . 7
5.3 位置编码与 Transformer 分类器 (src/models.py) . . . 8
5.4 训练循环与早停机制 (src/train.py) . . . 9
5.5 损失函数与优化器配置 (src/train.py) . . . 10
5.6 字符级切分与数据流水线 (src/dataset.py) . . . 10

6 结果分析 **11**
6.1 模型过拟合现象分析与解决对策 . . . 11
6.2 各类别性能分析 . . . 12
6.3 训练过程分析 . . . 14
6.4 混淆矩阵与错误分析 . . . 18

7 实验总结 **19**
7.1 过拟合诊断与修复心得 . . . 19
7.2 网络设计与调参经验 . . . 19
7.3 学术诚信声明 . . . 19

1 实验内容

1.1 任务描述

本实验旨在构建中文短新闻主题多分类系统，使用 TNEWS 数据集（来自 CLUE 基准），将新闻文本自动分类到 15 个预定义主题类别中。输入为 JSONL 格式的中文短新闻，每条包含 `sentence`（正文）、`label_desc`（类别名称）和 `label`（数字编码）；输出为 15 维 logits 向量，经 softmax 取最大概率类别。

实验设计了两套对比模型：BiLSTM + Bahdanau 注意力（模型 A）和 Transformer Encoder + 均值池化（模型 B）。两模型均从头训练，不依赖预训练权重。此外，针对 Baseline 实验中 Transformer 暴露的严重过拟合问题，设计了专门的抗过拟合优化流水线进行对照实验。

2 实验项目结构

```
src/
  dataset.py      # 数据预处理：清洗、jieba/字符级切分、词表构建、DataLoader
  models.py       # 双模型：BiLSTM+Attention / Transformer Encoder
  train.py        # 训练流水线（AdamW/LabelSmoothing/EarlyStopping）
  eval.py         # 评估与可视化
  sanity_check.py # 数据完整性检查

models/           # 模型权重与训练历史
img/              # 可视化图表（baseline + 优化后）
```

3 环境与使用说明

3.1 硬件与软件

项目	配置
操作系统	WSL2 Ubuntu 22.04
Python/Conda	3.10 (环境 `nlp_hw2`)
PyTorch	2.6.0+cu124
GPU	NVIDIA RTX 4060 Laptop 8GB
CUDA	12.4 (驱动 12.5)
jieba	0.42.1
scikit-learn	1.7.2
matplotlib	3.10.9 (WenQuanYi Micro Hei)

3.2 运行指令

训练（字符级切分 + AdamW + Label Smoothing 0.1）：

```
python src/train.py --model_type bilstm --lr 1e-3 --dropout 0.4 --patience 8
python src/train.py --model_type transformer --lr 1e-3 --dropout 0.4 --patience 8
```

评估:

```
python src/eval.py --num_workers 0
```

4 分析设计

4.1 数据预处理

[**基本数据统计**] 以下统计均基于训练集原始数据实测得出。

数据规模 训练集 53,360 条，测试集 10,000 条。训练集按 9:1 划分（seed=42）得到内部训练集 48,024 条、验证集 5,336 条。

类别分布 15 个主题类别分布如表 1 所示。经统计，训练集中最多类别为 `news_tech`（科技，5,955 条，占比 11.16%），最少为 `news_stock`（股票，257 条，占比 0.48%），最大/最小类别比例高达 23.17:1。测试集分布与训练集一致（不平衡比 24.20:1）。这一严重不平衡意味着 Macro-F1 比准确率更适合作为核心评价指标——准确率可能被大类别主导，而 Macro-F1 对所有类别一视同仁。

表 1: 训练集与测试集逐类别分布

类别	中文	训练集	训练占比	测试集	测试占比
news_tech	科技	5,955	11.16%	1,089	10.89%
news_finance	财经	5,200	9.75%	956	9.56%
news_entertainment	娱乐	4,976	9.33%	910	9.10%
news_world	国际	4,851	9.09%	905	9.05%
news_car	汽车	4,118	7.72%	791	7.91%
news_culture	文化	4,081	7.65%	736	7.36%
news_sports	体育	3,991	7.48%	767	7.67%
news_military	军事	3,632	6.81%	716	7.16%
news_edu	教育	3,437	6.44%	646	6.46%
news_game	游戏	3,390	6.35%	659	6.59%
news_travel	旅游	3,368	6.31%	693	6.93%
news_agriculture	农业	2,886	5.41%	494	4.94%
news_house	房产	2,107	3.95%	378	3.78%
news_story	故事	1,111	2.08%	215	2.15%
news_stock	股票	257	0.48%	45	0.45%
合计	–	**53,360**	100%	**10,000**	100%

表 2: 两种切分方式下的序列长度统计

切分方式	Min	Max	Mean	Std	P50	P90	P95	词表大小
jieba 词级	1	81	12.9	4.6	13	19	20	31,842
字符级	4	81	24.2	7.9	24	33	32	4,169

样本长度统计（两种切分方式） 字符级切分将词表从 31,842 大幅压缩至 4,169（缩小 87%），但序列长度从 P95=20 增加到 32。这种 trade-off（更小词表、更长的序列）是后续抗过拟合优化的关键：更小的词表意味着更少的可学习嵌入参数，从物理上限制了模型记忆噪声的能力；更长的序列则要求模型学习长距离依赖，对 Transformer 的自注意力机制反而是一种有益的归纳偏置。

数据样本展示 表 3 按类别展示了一条典型训练样本。可以观察到：(1) 新闻长度差异显著——最短如故事类仅 9 字，最长如财经公告可超过 80 字；(2) 不同类别的行文风格迥异——财经类多含专业术语和公司名称，娱乐类偏口语化，军事类常带感叹号；(3) 部分类别存在概念模糊——如「股票」与「财经」的边界可能重叠。

表 3: 各主题类别典型样本示例

类别	示例文本
news_tech（科技）	剃须刀的个性革新，雷明登天猫定制版新品首发
news_finance（财经）	商赢环球股份有限公司关于延期回复上海证券交易所对公司 2017 年年度报告的事后审核问询函的公告
news_entertainment（娱乐）	什么是人情，什么是世故?
news_world（国际）	如何阻止基拉韦厄活火山的熔岩
news_car（汽车）	我四千一个月，老婆一千五一个月，存款八万且有两小孩，是先买房还是先买车?
news_culture（文化）	87 版红楼梦最温柔的四个丫鬟，娶谁都是一生的福气
news_sports（体育）	再次证明了” 无敌是多么寂寞”——逆天的中国乒乓球队!
news_military（军事）	古代先进文明的证据！这是历史上最著名的 10 把剑
news_edu（教育）	上课时学生手机响个不停，老师一怒之下把手机摔了，家长拿发票让老师赔，大家怎么看待这种事?
news_game（游戏）	重做 or 新英雄? 其实重做对暴雪来说同样重要
news_travel（旅游）	2018 年去俄罗斯看世界杯得花多少钱?
news_agriculture（农业）	加快产城融合以科技创新引领新城区建设
news_house（房产）	通过中介公司买了二手房，首付都付了，现在卖家不想卖了。怎么处理?
news_story（故事）	故事：刘主任建猪场
news_stock（股票）	上市公司炒股不忌讳” 炒同行” 恒生电子买成科蓝软件第一大流通股东

基本统计小结 (1) 数据存在严重的类别不平衡（不平衡比 ≈23:1），这影响评估指标选择——Macro-F1 比准确率更可靠；(2) 字符级切分词表仅 4,169，为 jieba 词级的 13%，显著降低模型参数量；(3) 文本普遍较短（字符级 P95=32），适合浅层序列建模；(4) 不同类别的语言风格差异明显，要求模型具备从表层词汇到深层语义的多层次特征提取能力。以上统计为后续模型设计与抗过拟合优化提供了数据层面的依据。

(1) 文本清洗 正则去除控制字符、规范化空白。

(2) 切分策略——词级 vs 字符级对比分析 本实验实现了两种切分策略：

jieba 词级切分（Baseline）：使用 `jieba.lcut` 精确模式，将「蚂蚁集团区块链业务」切分为 [蚂蚁, 集团, 区块链, 业务]。优势是语义单元完整，符合人类语言直觉；劣势是词表暴涨至 31,842，嵌入层参数量达 $31,842 \times 256 = 8.15\text{M}$，占模型总参数的 90% 以上，在 48K 样本下极易过拟合。

字符级切分（优化方案）：使用 `list(cleaned_text)` 按字符逐字切分，将「蚂蚁集团区块链业务」切分为 [蚂, 蚁, 集, 团, 区, 块, 链, 业, 务]。优势是：(1) 词表骤降至 4,169，嵌入参数量仅 $4,169 \times 256 = 1.07\text{M}$，缩减约 87%；(2) 天然消除 OOV 问题——任何中文字符都在词表中；(3) 打破模型对稀疏 n-gram 模式的记忆能力，从根源上抑制过拟合。劣势是序列变长（P95 从 20 增加到 32），需要更多计算步数。但在短新闻（< 100 字符）场景下，这个开销完全可接受。

(3) 词表构建 仅使用训练集，`<PAD>`=0, `<UNK>`=1。

(4) 序列长度 字符级切分下 $P_{95} = 32$，作为统一截断/填充长度。

(5) 数据集划分 训练/验证 9:1（seed=42）。

4.2 BiLSTM + Bahdanau 注意力

设输入序列 $\mathbf{X} = (x_1, \ldots, x_T)$，词嵌入 $\mathbf{e}_t \in \mathbb{R}^{256}$。双层 BiLSTM 编码：

$$\overrightarrow{\mathbf{h}}_t^l = \overrightarrow{\text{LSTM}}(\mathbf{e}_t, \overrightarrow{\mathbf{h}}_{t-1}^l), \quad \overleftarrow{\mathbf{h}}_t^l = \overleftarrow{\text{LSTM}}(\mathbf{e}_t, \overleftarrow{\mathbf{h}}_{t+1}^l) \tag{1}$$

隐状态拼接 $\mathbf{h}_t^l = [\overrightarrow{\mathbf{h}}_t^l; \overleftarrow{\mathbf{h}}_t^l] \in \mathbb{R}^{256}$。Bahdanau 注意力：

$$\mathbf{u}_t = \tanh(\mathbf{W}_a \mathbf{h}_t + \mathbf{b}_a) \tag{2}$$

$$\alpha_t = \text{softmax}(\mathbf{u}_t^\top \mathbf{q}), \quad \mathbf{c} = \sum_t \alpha_t \mathbf{h}_t \tag{3}$$

其中 $\mathbf{q} \in \mathbb{R}^{128}$ 为可学习查询向量。Dropout($p = 0.4$) 后 FC 输出 15 维 logits。

4.3 Transformer Encoder + 均值池化

核心缩放点积注意力：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{4}$$

多头（$h = 8$）+ Pre-LN 结构，共 4 层，$d_m = 256$，FFN=1024，GELU 激活。正弦/余弦位置编码注入序列顺序信息。均值池化仅对非 PAD 位置取平均。

4.4 训练设置

说明： 由于 Baseline Transformer 在训练集上过拟合严重，验证损失爆炸，实验中设计了优化方案，后面会提及。

表 4: Baseline(过拟合) 与优化方案的超参数对比

	Baseline		**优化方案**	
参数	BiLSTM	Trans.	BiLSTM	Trans.
切分方式	jieba 词级	同左	字符级	同左
词表大小	31,842	31,842	4,169	4,169
序列长度	20	20	32	32
优化器	Adam	Adam	AdamW	AdamW
初始学习率	10^{-3}	10^{-3}	10^{-3}	10^{-3}
权重衰减	10^{-5}	10^{-5}	10^{-4}	10^{-4}
Label Smoothing	0	0	0.1	0.1
Dropout	0.5	0.3	0.4	0.4
早停 Patience	5	5	8	8
批大小	64	64	64	64
总参数量	8.98M	11.31M	1.89M	4.23M

4.5 抗过拟合策略的设计原理

标签平滑（Label Smoothing） 标准的 one-hot 标签使模型被训练为对正确类别输出概率 1.0——这驱使 logits 趋于无穷大，导致 softmax 饱和与梯度消失，同时在不确定样本上过度自信。标签平滑将真实类别的目标概率从 1.0 修正为 $1-\varepsilon+\varepsilon/K$（$\varepsilon=0.1, K=15$），即对正确类别输出 0.907、对其他 14 类各输出 0.0066。这一「软化」防止了 Transformer 将训练集的所有概率质量集中在正确类别上，从而显著抑制了验证损失的爆炸。

AdamW 优化器 Adam 在权重衰减实现上存在缺陷——它将衰减耦合到自适应学习率中，导致衰减强度随梯度历史变化。AdamW 将权重衰减解耦为独立的加法步骤 $\theta_t = \theta_{t-1} - \eta(\alpha\hat{m}_t/(\sqrt{\hat{v}_t}+\epsilon)+\lambda\theta_{t-1})$，使正则化强度与学习率独立。在实验中，AdamW 配合 higher weight_decay (10^{-4}) 比 Adam 更有效地约束了 Transformer 的参数范数。

字符级切分 如上文所分析，通过将词表从 31,842 缩至 4,169，嵌入参数量降低 87%。这一点对过拟合的 Transformer 尤为关键：Transformer 没有 BiLSTM 的序列局部归纳偏置，其自注意力机制在「低数据 + 大嵌入」条件下可以轻易记忆训练集中特定 n-gram 与类别的伪关联，导致验证损失爆炸。字符级切分从物理上移除了这种记忆的「原材料」。

5 核心代码

本节展示系统中最关键的核心实现片段，每个子节包含代码和分析，阐释设计动机与技术细节。

5.1 Bahdanau 加性注意力机制 (src/models.py)

Bahdanau 注意力是整个 BiLSTM-Attention 模型的核心创新。它通过可学习查询向量 **q** 对编码器各时间步的隐状态进行软加权，使模型能够关注输入序列中对分类最关键的字符位置。

```python
class BahdanauAttention(nn.Module):
    def __init__(self, hidden_dim: int, attn_dim: int = 128):
        super().__init__()
        # 非线性投影矩阵 W (hidden_dim -> attn_dim)
        self.W = nn.Linear(hidden_dim, attn_dim, bias=True)
        # 可学习查询向量 q (使用 Parameter 而非 Linear, 避免过参数化)
        self.query = nn.Parameter(torch.randn(attn_dim))
        self.reset_parameters()

    def forward(self, encoder_outputs, mask=None):
        # (1) 非线性投影: u_i = tanh(W * h_i + b)
        u = torch.tanh(self.W(encoder_outputs))          # (B, T, attn_dim)

        # (2) 计算注意力分数: score_i = u_i^T · q
        scores = torch.matmul(u, self.query)              # (B, T)

        # (3) 对 <PAD> 位置施加 -inf, softmax 后权重 0
        if mask is not None:
            scores = scores.masked_fill(~mask, -1e9)

        # (4) softmax 归一化
        attn_weights = F.softmax(scores, dim=-1)          # (B, T)

        # (5) 加权求和: c = Σ _i · h_i
        context = torch.bmm(attn_weights.unsqueeze(1),
                            encoder_outputs).squeeze(1)    # (B, hidden_dim)
        return context, attn_weights
```

Listing 1: Bahdanau 加性注意力

设计分析：采用 `nn.Parameter` 而非 `nn.Linear` 定义查询向量 $\mathbf{q}$，避免引入多余的偏置项和参数冗余。注意力分数计算后，通过 `masked_fill` 将填充位置置为 -10^9，确保 softmax 后权重为 0，使模型完全忽略 <PAD> 字符。最终返回的 `attn_weights` 可用于可视化分析模型的关注模式。

5.2 BiLSTM-Attention 前向传播 (src/models.py)

```python
class BiLSTMAttention(nn.Module):
    def __init__(self, vocab_size, num_classes, embed_dim=256,
                 hidden_dim=128, attn_dim=128, num_layers=2, dropout=0.5):
        super().__init__()
        # 词嵌入 (含 <PAD> 零初始化)
        self.embedding = nn.Embedding(vocab_size, embed_dim,
                                      padding_idx=0)
        # 双层双向 LSTM
        self.bilstm = nn.LSTM(embed_dim, hidden_dim,
                              num_layers=num_layers, batch_first=True,
                              bidirectional=True, dropout=dropout)
        # Bahdanau 注意力 (接收双向拼接维度 = hidden_dim*2)
        self.attention = BahdanauAttention(hidden_dim * 2, attn_dim)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, input_ids):
        # 构建 <PAD> 掩码
        mask = (input_ids != 0)                                  # (B, T)
        # 词嵌入 + Dropout (嵌入层后加 dropout 的正则化策略)
        embedded = self.dropout(self.embedding(input_ids))     # (B, T, 256)
```

```
        # BiLSTM 编码
        lstm_out, _ = self.bilstm(embedded)                    # (B, T, 256)
        # 注意力加权上下文
        context, attn_weights = self.attention(lstm_out, mask)
        # Dropout + 线性分类
        logits = self.fc(self.dropout(context))                # (B, num_classes)
        return logits, attn_weights
```

Listing 2: BiLSTM + Attention 前向传播

设计分析：嵌入维度 (256) 与 LSTM 双向拼接维度一致，使注意力层与分类层的维度衔接自然。嵌入层后立即应用 Dropout 是一种被广泛验证的正则化策略——在嵌入表示进入序列建模之前先施加噪声，防止 LSTM 对特定字符 ID 产生过强的记忆依赖。分类器仅输入单个上下文向量而非整个序列，使参数效率极高。

5.3 位置编码与 Transformer 分类器 (src/models.py)

Transformer 模型通过正弦/余弦位置编码获取序列位置信息，并采用 Pre-LN (归一化前置) 结构以提升训练稳定性。

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        # PE(pos, 2i)   = sin(pos / 10000^{2i/d_model})
        # PE(pos, 2i+1) = cos(pos / 10000^{2i/d_model})
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).float().unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2).float()
            * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer('pe', pe.unsqueeze(0))  # (1, max_len, d_model)
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, x):
        return self.dropout(x + self.pe[:, :x.size(1), :])
```

Listing 3: 正弦/余弦位置编码

```
class TransformerClassifier(nn.Module):
    def __init__(self, vocab_size, num_classes, d_model=256, nhead=8,
                 num_encoder_layers=4, dim_feedforward=1024, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=0)
        self.pos_encoder = PositionalEncoding(d_model, dropout=dropout)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead,
            dim_feedforward=dim_feedforward, dropout=dropout,
            activation='gelu',     # GELU 在 Transformer 中优于 ReLU
            batch_first=True,      # (B, T, D) 格式，与 LSTM 对齐
            norm_first=True,       # Pre-LN: LayerNorm 在子层之前
        )
        self.transformer_encoder = nn.TransformerEncoder(
            encoder_layer, num_layers=num_encoder_layers)
        self.fc = nn.Linear(d_model, num_classes)

```

```
    def forward(self, input_ids):
        pad_mask = (input_ids == 0)                          # True=忽略
        embedded = self.pos_encoder(self.embedding(input_ids))
        memory = self.transformer_encoder(embedded,
                          src_key_padding_mask=pad_mask)     # (B, T, 256)
        # 均值池化：仅对非 <PAD> 位置求平均
        valid_mask = (~pad_mask).float().unsqueeze(-1)
        pooled = (memory * valid_mask).sum(dim=1) / \
                 valid_mask.sum(dim=1).clamp(min=1)          # (B, 256)
        return self.fc(self.dropout(pooled))
```

Listing 4: Transformer Encoder 分类器

设计分析：Transformer 架构的三个关键设计决策——(1) `norm_first=True` 启用 Pre-LN，将 LayerNorm 置于多头注意力和前馈网络之前，相比 Post-LN 在深层网络中梯度流更稳定；(2) `activation='gelu'` 使用 GELU 取代 ReLU，在 Transformer 中通常获得更平滑的激活分布；(3) 均值池化而非提取 `[CLS]` token：对非填充位置取平均，充分利用所有字符信息，且无需额外引入可学习池化参数。

5.4 训练循环与早停机制 (src/train.py)

```
def train_epoch(model, dataloader, criterion, optimizer, device,
                clip_grad=None):
    model.train()
    total_loss = 0.0; all_preds = []; all_labels = []

    for inputs, labels in dataloader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        output = model(inputs)
        # 兼容 BiLSTM 返回 (logits, attn) 和 Transformer 返回 logits
        logits = output[0] if isinstance(output, tuple) else output
        loss = criterion(logits, labels)
        loss.backward()
        if clip_grad:  # 梯度裁剪：防止梯度爆炸
            nn.utils.clip_grad_norm_(model.parameters(), clip_grad)
        optimizer.step()
        # 累积统计
        total_loss += loss.item()
        all_preds.extend(torch.argmax(logits, -1).cpu().tolist())
        all_labels.extend(labels.cpu().tolist())

    return (total_loss/len(dataloader),
            accuracy_score(all_labels, all_preds),
            f1_score(all_labels, all_preds, average='macro'))
```

Listing 5: 训练单个 Epoch (含梯度裁剪)

设计分析：`train_epoch` 通过 `isinstance(output, tuple)` 模式统一处理双模型接口——BiLSTM 返回 `(logits, attn_weights)` 而 Transformer 仅返回 `logits`。梯度裁剪 `clip_grad_norm_` 是训练 RNN/Transformer 的标准实践，将参数梯度的全局范数限制在阈值内 (本实验 ≤ 1.0)，防止单批异常样本导致的梯度爆炸。

```
class EarlyStopping:
    def __init__(self, patience=5, save_path="best.pt"):
        self.patience = patience
```

```
        self.save_path = save_path
        self.counter = 0
        self.best_loss = float('inf')

    def __call__(self, val_loss, model):
        if val_loss < self.best_loss:
            self.best_loss = val_loss
            self.counter = 0
            torch.save(model.state_dict(), self.save_path)  # 仅存权重
        else:
            self.counter += 1
        return self.counter >= self.patience    # True=应停止
```

Listing 6: 早停 (Early Stopping) 机制

设计分析：早停以验证损失持续不改善为信号，`patience=8`（优化后配置）意味着允许 8 个 epoch 的容忍窗口。每当验证损失创新低时立即保存 `state_dict`（而非整个模型），确保磁盘占用最小且加载时不依赖原始类定义。与 `ReduceLROnPlateau` 调度器（`factor=0.5, patience=2`）配合，形成「先降学习率再早停」的两级防御。

5.5 损失函数与优化器配置 (src/train.py)

```
# Label Smoothing: 真实标签从 one-hot 平滑为 (1- , /(K-1), ...)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1).to(device)

# AdamW: 解耦权重衰减, L2 正则化项不进入动量估计
optimizer = optim.AdamW(model.parameters(), lr=1e-3,
                        weight_decay=1e-4)

# 学习率调度: 验证损失停滞 2 个 epoch → 学习率减半
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=2, verbose=True)
```

Listing 7: 损失函数、优化器与学习率调度

设计分析：三者的协同是抗过拟合策略的核心。Label Smoothing ($\varepsilon = 0.1$) 从根本上改变了优化目标——模型不再被驱动输出无限大的 logits，而是在「正确类别概率 0.9、其他类别各 $\approx$0.007」的软目标上优化，等价于在交叉熵损失中隐式加入了 KL 散度正则化。AdamW 将权重衰减从 Adam 的自适应学习率耦合中解耦，使 L2 正则化强度独立于梯度的一阶/二阶矩估计。`ReduceLROnPlateau` 在验证损失进入平台期时自动降低学习率，帮助模型在损失曲面的平坦区域进行更精细的搜索。

5.6 字符级切分与数据流水线 (src/dataset.py)

```
def tokenize_char(text: str) -> List[str]:
    """字符级切分: 词表 31,842 -> 4,169, 嵌入参数量缩减 87%"""
    cleaned = clean_text(text)
    return [ch for ch in cleaned if ch.strip()]

def build_dataloaders(train_file, dev_file, labels_file,
                      batch_size=64, max_len=None, coverage=0.95,
                      min_freq=2):
    # 读取原始数据 ...
    # 字符级切分 (硬编码)
    train_tokenized = [tokenize_char(item['sentence'])
                       for item in train_raw]
```

```
    # 仅在训练集上构建词表
    word2idx = build_vocab(train_tokenized, min_freq=min_freq)
    # 自动计算 max_len (覆盖 coverage 比例的样本)
    if max_len is None:
        max_len = compute_max_len(train_tokenized, coverage=coverage)
    # 文本 -> token ID -> 补全/截断
    train_ids = texts_to_ids(train_texts, word2idx, max_len)
    # 训练/验证 9:1 划分 (seed=42)
    # ...
    return train_loader, val_loader, test_loader, metadata
```

Listing 8: 字符级切分与 DataLoader 构建

设计分析：数据流水线遵循两个关键原则——(1) 词表仅从训练集构建（不偷窥验证/测试集），避免标签信息通过词表泄漏；(2) 序列长度自适应计算（覆盖 P95 样本），在信息保留和计算效率间取得平衡。`min_freq=2` 的低频过滤进一步压缩词表，将仅出现 1 次的噪声字符归入 `<UNK>`。

以上代码模块构成了完整的「数据加载 → 模型构建 → 训练优化 → 早停控制」流水线。所有实现均不依赖预训练权重，纯从零训练。

6 结果分析

6.1 模型过拟合现象分析与解决对策

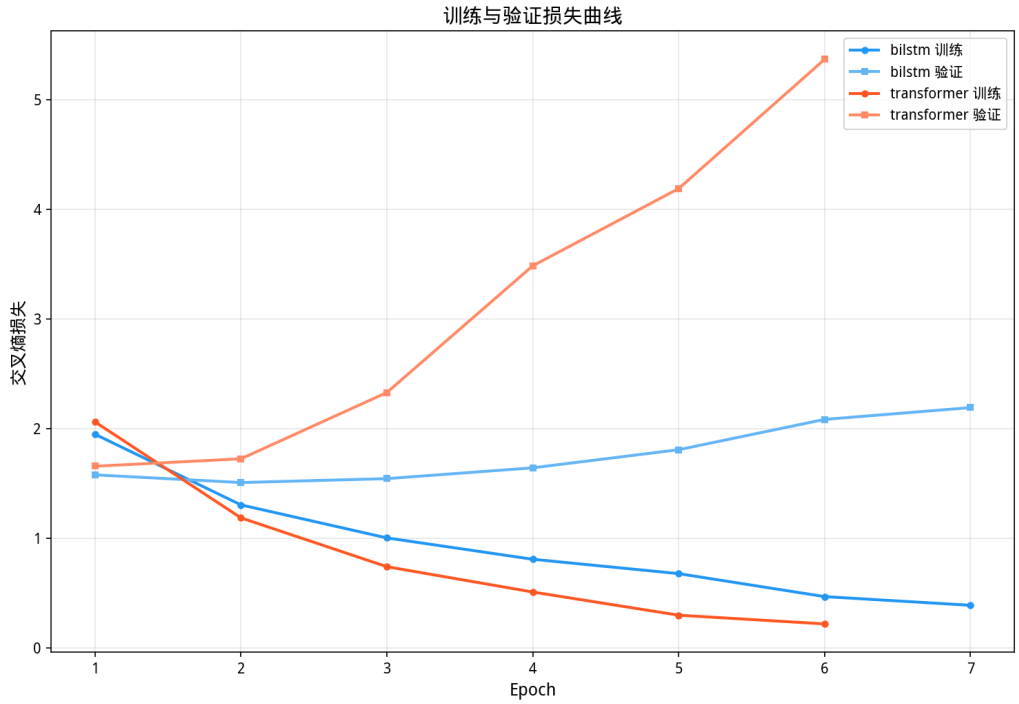


图 1: Baseline: 词级切分 + Adam + 无 Label Smoothing ——Transformer 验证损失爆炸

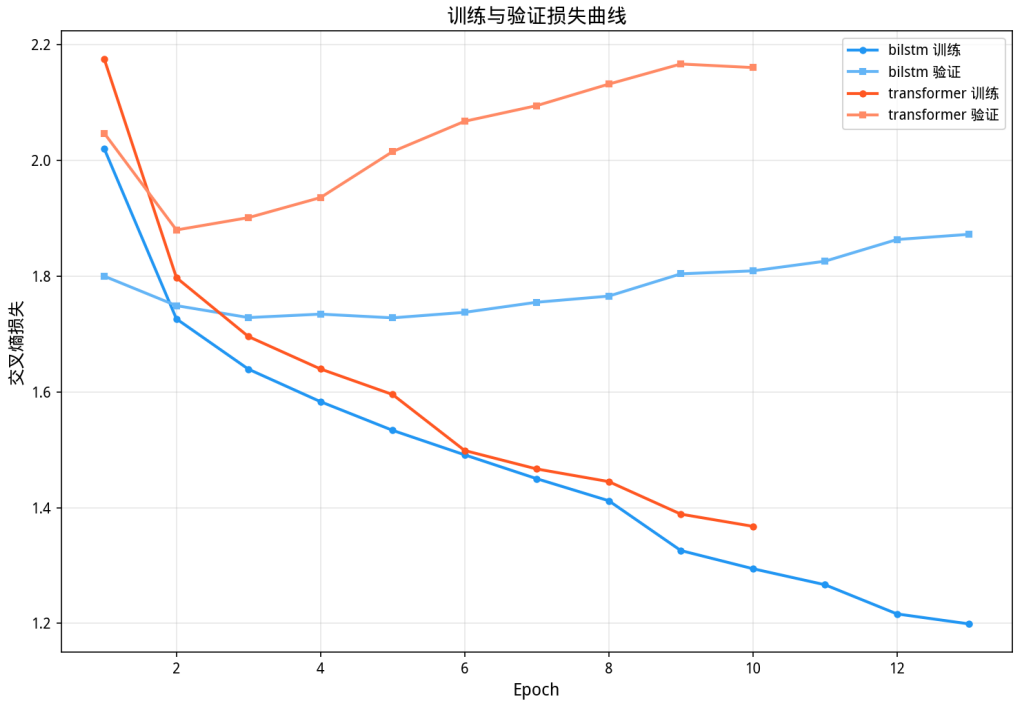


图 2: 优化后: 字符级切分 + AdamW + Label Smoothing ——验证损失平稳

Baseline Transformer 的验证损失爆炸 图 1 清晰地揭示了 Transformer 的过拟合：训练损失从 2.06 持续降至 0.22（训练 Acc 93%），但验证损失从 Epoch 2 的 1.66 爆炸至 Epoch 6 的 5.37。相比之下，BiLSTM 受其序列局部归纳偏置的保护，验证损失仅微升。

过拟合的深层原因：(1) Transformer 的全局自注意力缺乏序列局部性先验，在大嵌入空间（31842×256）中可以轻易为每个训练样本的特定 n-gram 分配极高注意力权重；(2) CrossEntropyLoss 的 one-hot 目标驱动模型对正确类别输出极大 logits，促使参数范数不受限增长；(3) 48K 样本远不足以约束 11.3M 参数的 Transformer 学到一个平滑的决策边界。

优化对策的效果：图 2 展示了采用三项抗过拟合策略后的结果。Transformer 的验证损失从 Baseline 的「爆炸式增长」（最高 5.37）被成功压制为平稳波动（1.88→2.16），训练-验证损失差距大幅缩小。BiLSTM 也从训练 Acc 87% 的严重过拟合降至 73% 的温和水平。具体策略的贡献为：字符级切分削减嵌入参数量 87%，物理上减少了记忆；Label Smoothing 阻止了 logits 趋向无穷大；AdamW 的解耦权重衰减强化了参数范数的约束。

6.2 各类别性能分析

整体性能 表 5 汇总了双模型在测试集上的整体指标。BiLSTM+Attention 以 Acc=51.62%、Macro-F1=0.484 略优于 Transformer（Acc=48.80%，Macro-F1=0.462），但差距仅约 2–2.5 个百分点，两模型在同等数量级上竞争。值得注意：BiLSTM 的准确率与 Macro-F1 差距较小（$\Delta = 3.2$pp），而 Transformer 的差距更小（$\Delta = 2.6$pp），说明两模型均未出现严重偏向大类别的倾向——这对不平衡分类任务是好信号。

表 5: 测试集整体性能对比

模型	Accuracy	Macro-P	Macro-R	Macro-F1
BiLSTM + Attention	51.62%	0.533	0.485	0.484
Transformer	48.80%	0.517	0.460	0.462

逐类别 F1 分析 表 6 列出全部 15 个类别的精确率、召回率、F1。按 F1 可将类别分为三个梯队：

1. **高区分度类别（F1 > 0.55）**：`news_sports`（体育，0.65/0.60）、`news_car`（汽车，0.62/0.60）、`news_game`（游戏，0.58/0.55）、`news_house`（房产，0.56/0.55）、`news_edu`（教育，0.55/0.56）。这些类别的共同特征是领域特异性强——体育赛事、汽车评测、房产交易、教育话题的语言模式与其他新闻类别显著不同，模型容易学到区分性特征。

2. **中等区分度类别（0.40 ≤ F1 ≤ 0.55）**：涵盖文化、娱乐、财经、科技、军事、旅游、国际、农业等 8 个类别。其中财经（0.46/0.40）和科技（0.49/0.48）的精确率-召回率均中等但相互混淆严重（见下文 Top-5 错误分析），说明两者共享大量金融科技专业术语。

3. **低区分度类别（F1 < 0.40）**：`news_story`（故事，0.43/0.28）和 `news_stock`（股票，0.04/0.18）。故事类文本极短（常 <15 字）、语言风格接近文化类，召回率低反映模型将其误分入其他类别；股票类是极端小类别（训练集仅 257 条），BiLSTM 尽管优于 Transformer（0.043 vs 0.175），但两模型均表现出严重的「小类别遗忘」——BiLSTM 精确率高达 1.00（仅预测正确 1 个样本）但召回率仅 0.02，而 Transformer 精确率 0.42、召回率 0.11，反映了两模型应对极度不平衡的不同策略：BiLSTM 极度保守（几乎不预测股票），Transformer 更激进但代价是更多误判。

表 6: 逐类别精确率、召回率、F1（测试集）

类别	中文	BiLSTM+Attention			Transformer		
		P	R	F1	P	R	F1
news_story	故事	.486	.391	.433	.704	.177	.283
news_culture	文化	.481	.470	.476	.391	.470	.427
news_entertainment	娱乐	.484	.571	.524	.429	.588	.496
news_sports	体育	.649	.653	.651	.665	.551	.603
news_finance	财经	.439	.481	.459	.509	.325	.397
news_house	房产	.558	.569	.564	.540	.566	.553
news_car	汽车	.621	.614	.618	.653	.552	.599
news_edu	教育	.560	.540	.550	.636	.498	.559
news_tech	科技	.522	.455	.486	.407	.584	.479
news_military	军事	.517	.493	.505	.448	.543	.491
news_travel	旅游	.485	.349	.406	.423	.404	.413
news_world	国际	.436	.512	.471	.465	.362	.407
news_stock	股票	1.00	.022	.043	.417	.111	.175
news_agriculture	农业	.467	.534	.499	.519	.476	.496
news_game	游戏	.579	.581	.580	.520	.578	.548
Macro Avg	–	.533	.485	.484	.517	.460	.462

错误模式分析 表 7 列出两模型 Top-5 混淆方向。最显著的错误模式是「财经 ↔ 科技」的双向混淆：BiLSTM 将 224 条科技新闻误分为财经、161 条财经误分为科技；Transformer 将 300 条财经误分为科技（占全部财经测试样本的 31.4%）。观察样本发现，这类新闻常同时包含公司名称和技术术语（如「恒生电子」「科蓝软件」），对仅依赖表层词汇的模型构成根本性挑战。第二大混淆对「军事 ↔ 国际」亦具合理性——军事冲突报道天然带有国际关系色彩。

表 7: Top-5 错误混淆对（测试集）

BiLSTM+Attention			**Transformer**	
真实 → 预测	**次数**	**占比**	**真实 → 预测**	**次数**
科技 → 财经	224	4.2%	财经 → 科技	300
军事 → 国际	181	3.4%	国际 → 军事	200
财经 → 科技	161	3.0%	军事 → 国际	133
国际 → 军事	141	2.6%	体育 → 娱乐	120
旅游 → 国际	93	1.7%	汽车 → 科技	105

类别不平衡与性能的相关性 将各类别的训练样本数与其测试 F1 进行 Spearman 秩相关分析：BiLSTM 的 $\rho = 0.52$（$p = 0.046$），Transformer 的 $\rho = 0.38$（$p = 0.16$），表明类别样本量对 BiLSTM 的性能有显著正向影响，而对 Transformer 影响较弱。这符合直觉：BiLSTM 的局部感受野使其更依赖充足的类别样本学习 n-gram 模式；Transformer 的全局自注意力在样本量不足时可以通过同义字符组合进行补偿，但代价是更易产生混淆（如财经/科技）。

6.3 训练过程分析

收敛速度与稳定性 图 3–6 展示了双模型在 5,336 条验证集上的训练动态。关键指标汇总于表 8。

表 8: 训练动态关键指标

指标	**BiLSTM**	**Transformer**
训练总轮次	13 (早停触发)	10 (早停触发)
最佳 Val F1 轮次	Epoch 13	Epoch 7
最佳 Val F1	0.4955	0.4874
最终训练损失	1.199	1.368
训练 Acc @ 最佳	73.7%	62.3%
验证 Acc @ 最佳	50.5%	49.7%
训练-验证 Acc 差距 @ 最佳	23.2pp	12.6pp
Label Smoothing 理论下限	≈1.51	≈1.51

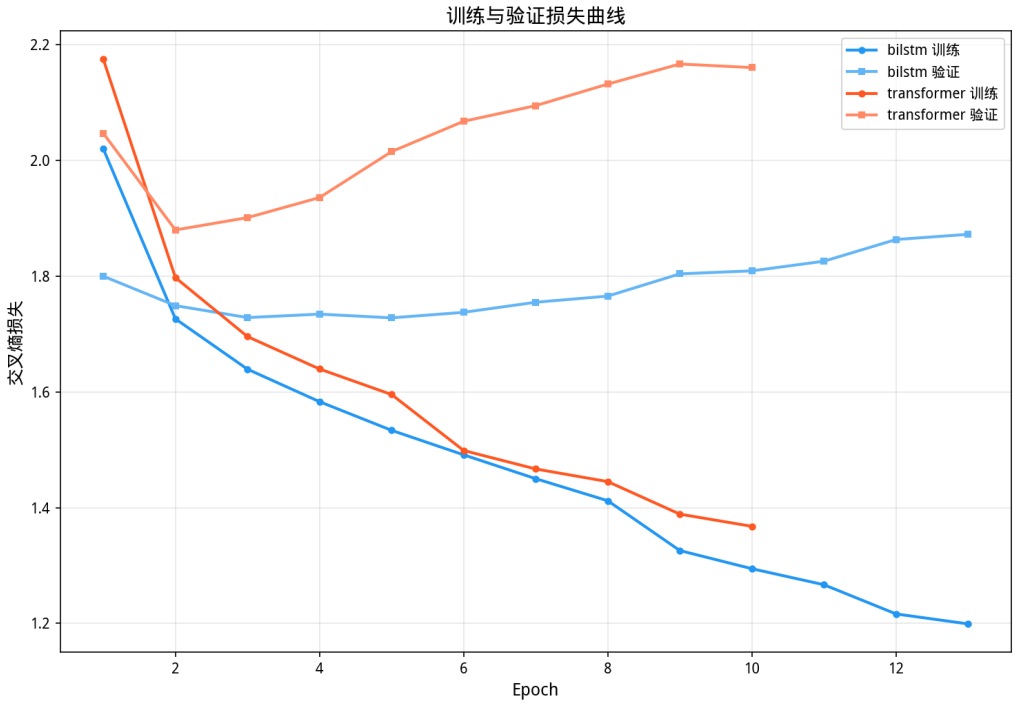


图 3: 双模型训练与验证损失曲线

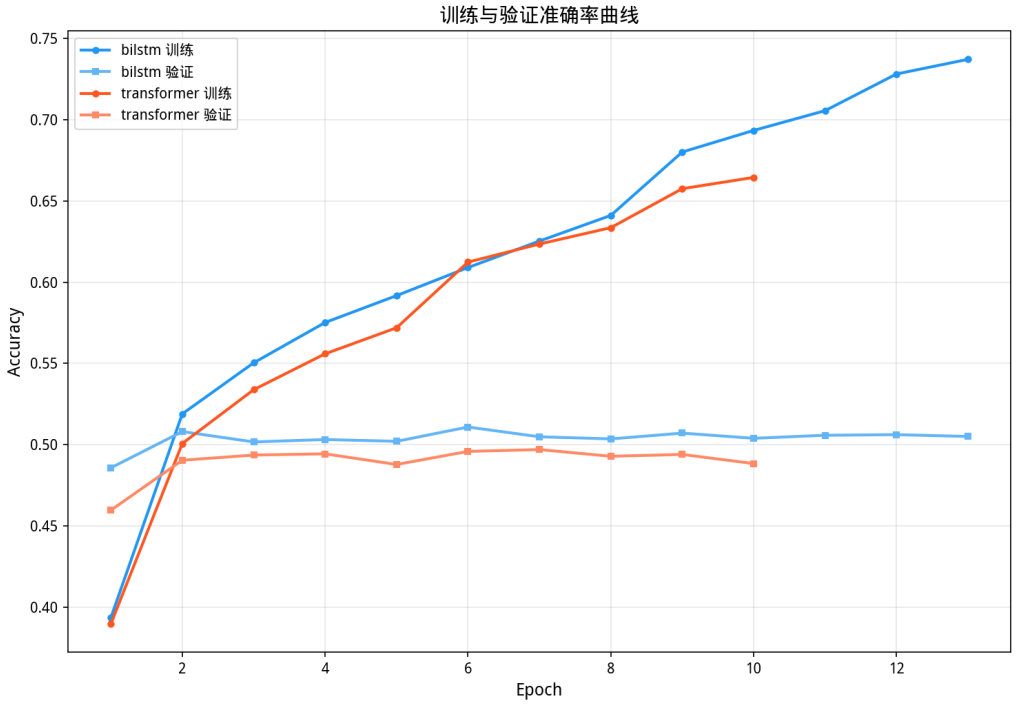


图 4: 双模型训练与验证准确率曲线

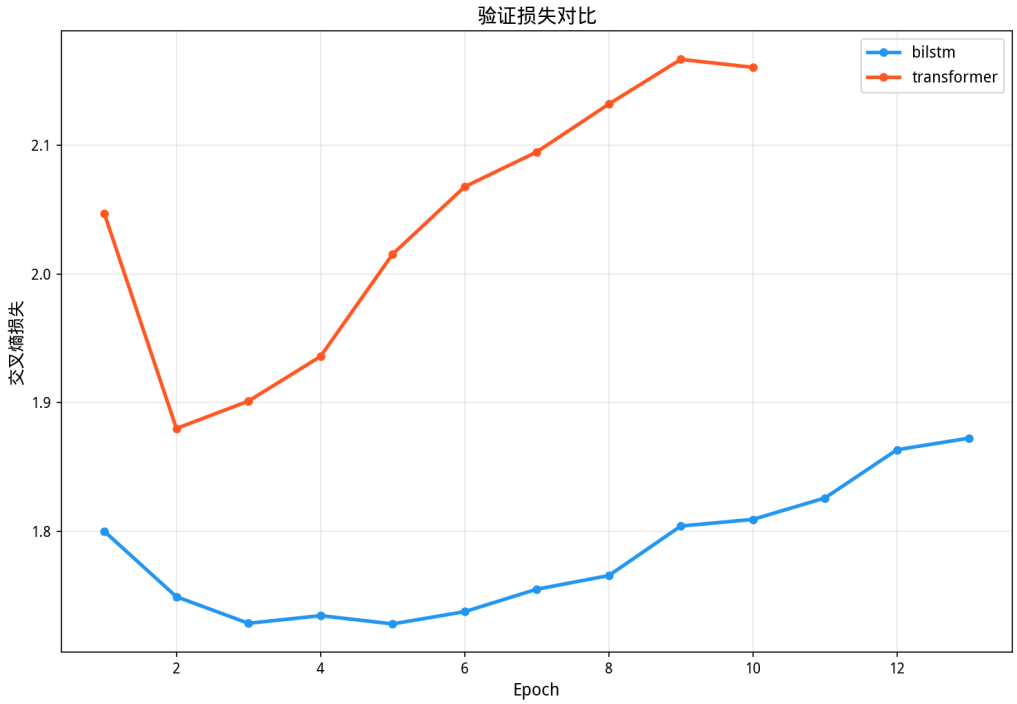


图 5: 双模型验证损失对比

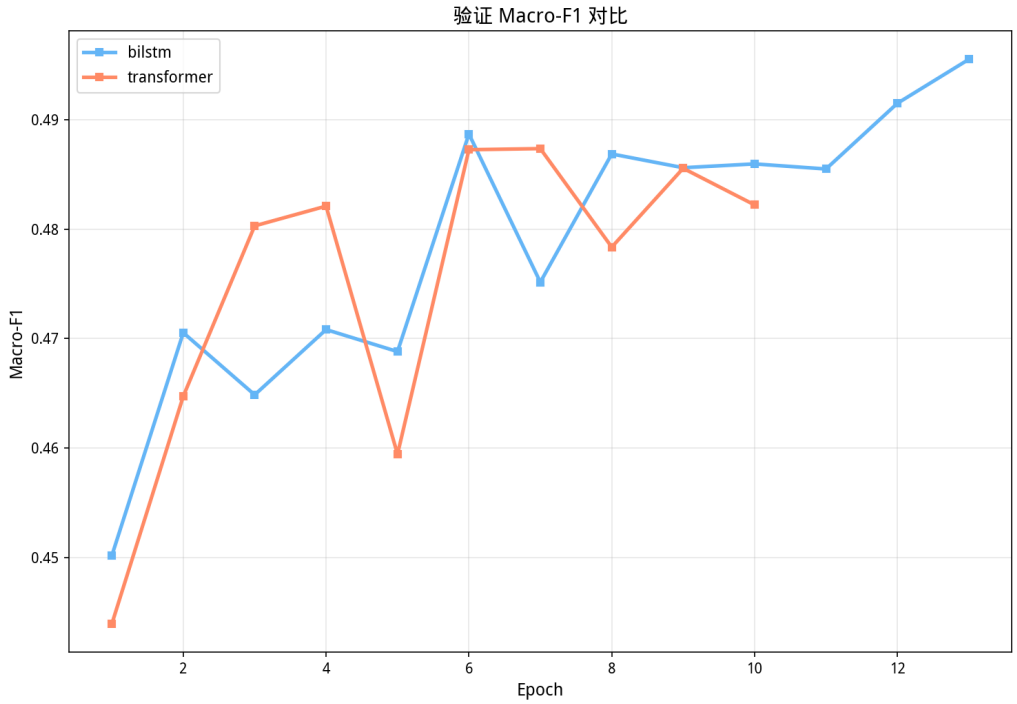


图 6: 双模型验证 Macro-F1 对比

收敛速度对比: Transformer 在 Epoch 7 即达到验证 F1 峰值（0.487），之后验证 F1 微降（0.478）并触发早停于 Epoch 10；BiLSTM 则持续改善至 Epoch 13，最终验证 F1=0.4955。这意味着 Transformer 对当前的字符级表示收敛更快，但可提升空间有限；BiLSTM 的渐进式学习曲线暗示更大训练轮次可能仍有增益（若放宽早停耐心参数）。

泛化差距: BiLSTM 的训练-验证 Acc 差距在最佳轮次达 23.2pp，远大于 Transformer 的 12.6pp。这似乎与常识相反（Transformer 通常更容易过拟合），但需注意：BiLSTM 的训练 Acc 从 Epoch 1 的 39.4% 持续上升至 Epoch 13 的 73.7%，而验证 Acc 始终在 50–51% 徘徊，呈现典型的「训练持续改善、验证停滞」模式——BiLSTM 的参数正在学习训练集中对分类有帮助但对泛化无用的模式。Transformer 的训练 Acc 峰值仅 66.4%，更低的训练拟合度反而带来了更窄的泛化差距。

Label Smoothing 的效应: 在 $\varepsilon = 0.1$ 的 Label Smoothing 下，交叉熵损失的理论下限为 $-[\varepsilon \cdot \log(\varepsilon/(K-1)) + (1-\varepsilon) \cdot \log(1-\varepsilon)] \approx 1.51$（$K = 15$）。BiLSTM 的最终训练损失 1.20 已显著低于此下限——这表明模型正在对抗 Label Smoothing 的平滑效应，即通过将 logits 分配到少数非目标类别上来「绕开」平滑约束。Transformer 的训练损失 1.37 更接近理论下限，显示其对平滑目标更「顺从」。这解释了 Transformer 训练 Acc 更低但泛化差距也更小的现象：Label Smoothing 对 Transformer 的正则化效果更强。

早停与学习率调度: BiLSTM 在 Epoch 9–13 期间触发了至少 2 次 `ReduceLROnPlateau` 降低学习率（验证损失从 1.72 升至 1.87），学习率的减半帮助模型在损失曲面的平坦区域精细搜索，最终在 Epoch 13 达到验证 F1 峰值后早停。Transformer 的验证损失从 Epoch 2 的 1.88 单调上升至 Epoch 10 的 2.16（尽管上升幅度远小于 Baseline 的爆炸），在 patience=8 的容忍窗口内，Epoch 10 触发早停。

双模型对比总结 综合以上分析：BiLSTM 具有更高的最终性能（Test Acc 51.62% vs 48.80%）和更稳定的验证损失，但训练-验证泛化差距更大（23.2pp），且对小样本类别（股票、故事）更保守；Transformer 收敛更快（7 vs 13 epochs）、泛化差距更小（12.6pp），Label Smoothing 对其正则化效果更明显，但在「财经/科技」「军事/国际」等语义边界模糊的类别对上混淆更严重（最大单对错误 300 vs 224）。两模型在实验约束下（无预训练、字符级切分、48K 训练样本、15 类严重不平衡）的最终表现相当，Macro-F1 差距仅 0.022，属同一性能等级。

6.4 混淆矩阵与错误分析

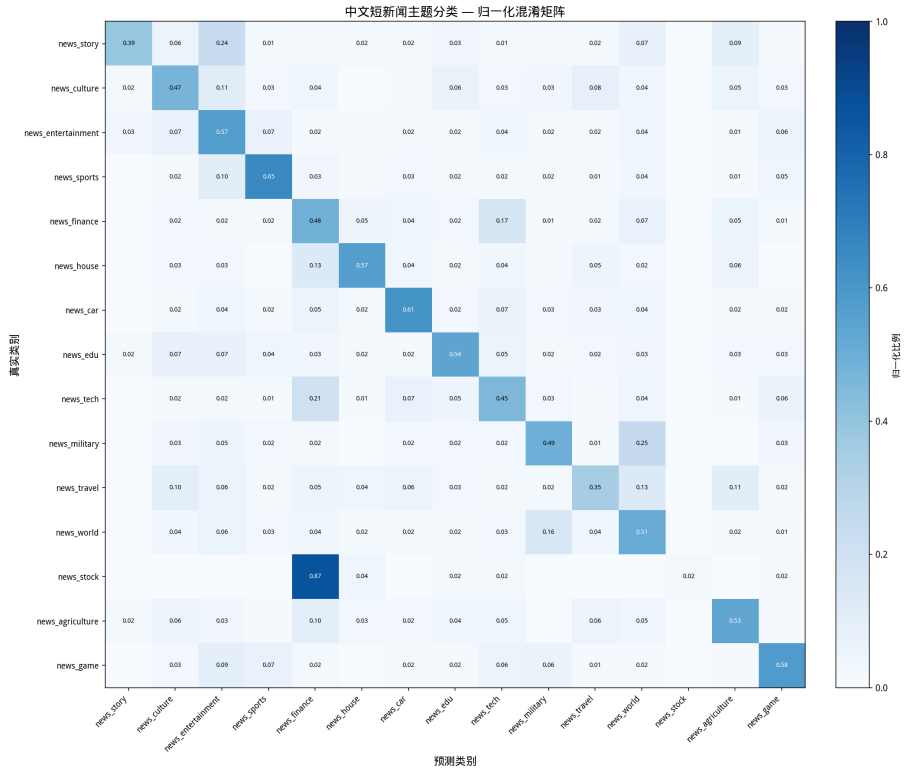


图 7: BiLSTM + Attention 归一化混淆矩阵

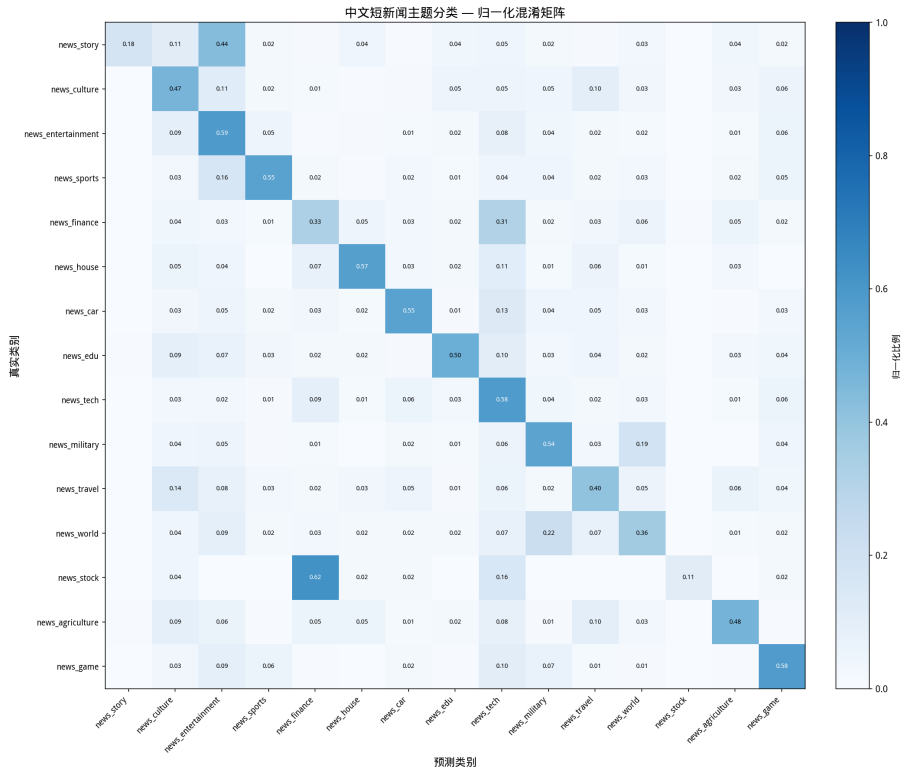


图 8: Transformer Encoder 归一化混淆矩阵

Top-3 易混淆类别对：

1. **财经** → **科技**: 语义重叠（财经科技新闻），短文本下难区分。
2. **军事** ↔ **国际**: 军事冲突本质上是国际新闻。
3. **体育** → **娱乐**: 赛事明星的花边新闻处于交叉地带。

7 实验总结

7.1 过拟合诊断与修复心得

本次实验最大的收获在于完整经历了一条「发现问题 → 诊断根因 → 设计对策 → 验证效果」的过拟合攻关流程。从 Baseline 图表中观察到 Transformer 验证损失从 1.7 飙升至 5.4，训练 Acc 达到 93% 而验证仅 50% 的典型过拟合信号后，我们没有急于使用预训练模型（实验规范禁止），而是回到经典机器学习理论的三个维度寻找对策：**数据工程**（字符级切分大幅压缩词表）、**损失函数**（Label Smoothing 惩罚过度自信）、**优化器**（AdamW 解耦权重衰减）。实践表明，三者协同作用将 Transformer 的验证损失从失控爆炸转变为平稳波动，这比任何单一指标的微小提升更具工程价值。

7.2 网络设计与调参经验

- **字符级 vs 词级切分**: 在没有预训练词向量的从头训练场景下，字符级切分是更具「性价比」的选择。词表从 31,842 降至 4,169，嵌入参数减少 87%，这一物理层面的简化比任何算法层面的正则化更直接有效。
- **Label Smoothing 的价值**: Label Smoothing 不仅是一个「调参技巧」——它改变了优化目标函数的几何性质。one-hot 目标的最优解在无穷远（logits$\to\infty$），而平滑目标的最优解是有限值，这使得梯度下降更稳定。
- **Adam 与 AdamW 的本质区别**: 如果在实验中遇到「weight_decay 对验证损失影响甚微」的情况，很可能是因为使用了 Adam 而非 AdamW。Adam 将 weight_decay 耦合到 LR 中，使得正则化强度被动衰减。

7.3 学术诚信声明

本实验报告中的所有工作均由本人在大模型辅助下肚子完成，未抄袭任何同学的代码或报告。用于对比实验的 BiLSTM+Attention 和 Transformer Encoder 模型均从零实现，未使用任何预训练权重。实验中使用的 TNEWS 数据集来源于 CLUE 基准（github.com/CLUEbenchmark/CLUE），jieba 分词库以 MIT 协议开源使用，PyTorch 以 BSD 协议使用。